Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of

Electrical & Computer Engineering

Lab Report 6

Design and Implementation of Error Detection and Correction Mechanisms using CRC and Hamming Codes

Course Code: ECE 4212

Course Title: Computer Network Sessional

Submitted to:
Md. Faysal Ahamed
Assistant Professor
Dept of ECE, RUET

Submitted by:
Md. Tajim An Noor
Roll: 2010025
Semester: 4th Year Even

Submission Date: April 19, 2026

Contents

Design and Implementation of Error Detection and Correction Mechanisms using CRC and Hamming Codes

1 Theory and Introduction

In digital communication systems, data transmitted over noisy channels or stored in unreliable media is susceptible to bit errors. Error detection and correction codes are essential mechanisms employed to ensure data integrity [**?**]. This experiment focuses on two fundamental coding techniques: Cyclic Redundancy Check (CRC) for error detection and Hamming Code for single-bit error correction.

Cyclic Redundancy Check (CRC) is a widely used error-detecting code that appends a short check value (redundancy bits) to a data block. It treats binary data as a polynomial and uses modulo-2 arithmetic (XOR operations) for division [1]. A generator polynomial (key) is agreed upon by both sender and receiver. It is highly effective at detecting burst errors and is commonly used in network protocols like Ethernet.

Hamming Code, developed by Richard Hamming, is an error-correcting code capable of detecting and correcting single-bit errors [2]. This is achieved by inserting multiple parity bits at positions that are powers of two (1, 2, 4, 8, etc.). The number of redundant bits r required for m data bits must satisfy the condition: $2^r \geq m + r + 1$. By recalculating these parity bits at the receiver, the exact position of an erroneous bit can be isolated and rectified [3].

2 Methodology and Implementation

The experiment was conducted by implementing software simulations of the CRC and Hamming Code algorithms in Python. The following processes were executed:

2.1 CRC Implementation Strategy

- **Encoding (Sender):** The original data was appended with $k - 1$ zeros, where k is the length of the generator polynomial. Modulo-2 division was performed on the augmented data using the generator polynomial. The resulting remainder was appended to the original data to form the transmitted codeword.
- **Decoding (Receiver):** Modulo-2 division was performed on the received codeword using the same polynomial. If the final remainder evaluated to zero, the data was accepted as error-free; otherwise, an error was flagged.

2.2 Hamming Code Implementation Strategy

- **Encoding:** The number of required redundant bits was calculated. The data bits were placed in non-power-of-two positions. Each parity bit was calculated to ensure even parity across its designated checking positions.
- **Correction:** At the receiver, all parity bits were recalculated. The XOR of the differing parity positions yielded the error location (syndrome), which was subsequently flipped to restore the original data.

2.3 CRC Simulation Code

The Python code utilized to simulate the detailed, step-by-step operations if CRC is provided below:

```python
def findXor(a, b):
    """Performs bitwise XOR between two binary strings (a and b)."""
    result = []
    for i in range(1, len(b)):  # Skip first bit (CRC standard)
        if a[i] == b[i]:
            result.append("0")
        else:
            result.append("1")
    return "".join(result)


def mod2div(dividend, divisor, show_steps=False):
    """Performs Modulo-2 division (CRC division algorithm) with step tracking."""
    pick = len(divisor)
    tmp = dividend[0:pick]

    if show_steps:
        print(f"    [DIV] Initial Dividend: {dividend}")
        print(f"    [DIV] Divisor: {divisor}")

    while pick < len(dividend):
        if tmp[0] == "1":
            if show_steps:
                print(f"    [DIV] {tmp} XOR {divisor} -> ", end="")
            tmp = findXor(divisor, tmp) + dividend[pick]
            if show_steps:
                print(f"{tmp} (pulled down {dividend[pick]})")
        else:
            if show_steps:
                print(f"    [DIV] {tmp} XOR {'0'*pick} -> ", end="")
            tmp = findXor("0" * pick, tmp) + dividend[pick]
            if show_steps:
                print(f"{tmp} (pulled down {dividend[pick]})")
        pick += 1
    # Final step
    if tmp[0] == "1":
        if show_steps:
            print(f"    [DIV] Final Step: {tmp} XOR {divisor} -> ", end="")
        tmp = findXor(divisor, tmp)
    else:
        if show_steps:
            print(f"    [DIV] Final Step: {tmp} XOR {'0'*pick} -> ", end="")
        tmp = findXor("0" * pick, tmp)

    if show_steps:
        print(f"{tmp}\n")
    return tmp


def encodeData(data, key):
    """Appends CRC remainder to the original data."""
    l_key = len(key)
    appended_data = data + "0" * (l_key - 1)
    print(f"--- SENDER SIDE ---")
    print(f"Original Data: {data}")
    print(f"Key (Polynomial): {key}")
    print(f"Appended Data (with {l_key - 1} zeros): {appended_data}\n")

    print("Calculating Remainder:")
    remainder = mod2div(appended_data, key, show_steps=True)

    codeword = data + remainder
    print(f"Calculated Remainder: {remainder}")
    print(f"Transmitted Codeword: {codeword}\n")
    return codeword


def receiver(data, key):
    """Checks if received data has errors (remainder == 0)."""
```

```
    print(f"--- RECEIVER SIDE ---")
    print(f"Received Codeword: {data}\n")

    print("Performing Modulo-2 Division on Received Data:")
    remainder = mod2div(data, key, show_steps=True)

    print(f"Final Remainder: {remainder}")
    if "1" in remainder:
        print(
            "[RESULT] ERROR DETECTED: The remainder is non-zero. The data was corrupted during transmission.\n"
        )
    else:
        print("[RESULT] SUCCESS: The remainder is zero. No errors detected.\n")


if __name__ == "__main__":
    data = "100100"
    key = "1101"
    # 1. Sender Encodes
    codeword = encodeData(data, key)
    # 2. Receiver Receives Valid Data
    print(">>> SCENARIO 1: Transmission without errors")
    receiver(codeword, key)
    # 3. Receiver Receives Corrupted Data
    print(">>> SCENARIO 2: Transmission WITH errors (Fault Simulation)")
    # Flip the 3rd bit from '0' to '1'
    error_codeword = codeword[:2] + "1" + codeword[3:]
    print(f"(Simulating noise... flipping a bit: {codeword} -> {error_codeword})")
    receiver(error_codeword, key)
```

2.4 Hamming Code Simulation Code

The Python code utilized to simulate the detailed, step-by-step operations if Hamming Code is provided below:

```
def calcRedundantBits(m):
    """Calculate the number of redundant bits needed."""
    for i in range(m):
        if 2**i >= m + i + 1:
            return i


def posRedundantBits(data, r):
    """Place redundant bits at positions that are powers of two."""
    j = 0
    k = 1
    m = len(data)
    res = ""
    for i in range(1, m + r + 1):
        if i == 2**j:
            res = res + "0"
            j += 1
        else:
            res = res + data[-1 * k]
            k += 1
    return res[::-1]


def calcParityBits(arr, r, show_steps=False):
    """Calculate and set parity bits based on bit positions."""
    n = len(arr)
    if show_steps:
        print("Calculating Parity Bits (Even Parity):")

    for i in range(r):
        val = 0
        checked_positions = []
        for j in range(1, n + 1):
            if j & (2**i) == (2**i):
                val = val ^ int(arr[-1 * j])
                checked_positions.append(f"pos {j} (val {arr[-1 * j]})")
        # Insert the calculated parity bit back into the array
        arr = arr[: n - (2**i)] + str(val) + arr[n - (2**i) + 1 :]

        if show_steps:
            print(f"  P{2**i} checks: {', '.join(checked_positions)}")
            print(f"  => P{2**i} is set to {val}")

    return arr


def detectAndCorrectError(arr, r):
    """Detect error position using parity bits and correct it."""
    n = len(arr)
    res = 0
    print("\n--- RECEIVER SIDE: Error Checking ---")
    print("Recalculating parity bits over received data:")

    for i in range(r):
```

```
        val = 0
        for j in range(1, n + 1):
            if j & (2**i) == (2**i):
                val = val ^ int(arr[-1 * j])
        res = res + val * (10**i)
        print(f"  P{2**i} parity check evaluates to: {val}")
    # Convert binary syndrome to decimal
    error_pos = int(str(res), 2)

    if error_pos == 0:
        print("\n[RESULT] Syndrome is 0. No error in the received message.")
        return arr
    else:
        print(
            f"\n[RESULT] Syndrome is {str(res).zfill(r)}. Error detected at position {error_pos} (from the right)."
        )
        # Correct the error
        arr_list = list(arr)
        error_index = n - error_pos
        print(
            f"-> Fixing error... Flipping bit at index {error_index} (which is currently '{arr_list[error_index]}')"
        )

        arr_list[error_index] = "0" if arr_list[error_index] == "1" else "1"
        corrected_arr = "".join(arr_list)
        print(f"-> Corrected Data Codeword: {corrected_arr}")
        return corrected_arr


if __name__ == "__main__":
    data = "1011001"
    m = len(data)
    r = calcRedundantBits(m)

    print("--- SENDER SIDE ---")
    print(f"Original Data: {data}")
    print(f"Data Bits (m): {m}")
    print(f"Redundant Parity Bits required (r): {r}")
    # Place '0' in parity positions (1, 2, 4, 8)
    arr = posRedundantBits(data, r)
    print(f"Data with empty parity bits: {arr}")
    # Calculate actual parity
    arr = calcParityBits(arr, r, show_steps=True)
    print(f"\nTransmitted Codeword: {arr}")
    # Simulate Error
    print("\n>>> Simulating transmission noise...")
    # Let's flip the bit at position 7 from the right (index 4 from left)
    error_arr = arr[:4] + ("0" if arr[4] == "1" else "1") + arr[5:]
    print(f"Original sent: {arr}")
    print(f"Received     : {error_arr}")

    # Detect and correct
    corrected_data = detectAndCorrectError(error_arr, r)
```

3 Results

The implemented Python algorithms were executed to simulate data transmission, error injection, and detection/correction. The verbose console outputs are documented below.

3.1 CRC Simulation Output

```
--- SENDER SIDE ---
Original Data: 100100
Key (Polynomial): 1101
Appended Data (with 3 zeros): 100100000

Calculating Remainder:
    [DIV] Initial Dividend: 100100000
    [DIV] Divisor:          1101
    [DIV] 1001 XOR 1101 -> 1000 (pulled down 0)
    [DIV] 1000 XOR 1101 -> 1010 (pulled down 0)
    [DIV] 1010 XOR 1101 -> 1110 (pulled down 0)
    [DIV] 1110 XOR 1101 -> 0110 (pulled down 0)
    [DIV] 0110 XOR 00000000 -> 1100 (pulled down 0)
    [DIV] Final Step: 1100 XOR 1101 -> 001
```

```

Calculated Remainder: 001
Transmitted Codeword: 100100001

>>> SCENARIO 1: Transmission without errors
--- RECEIVER SIDE ---
Received Codeword: 100100001

Performing Modulo-2 Division on Received Data:
    [DIV] Initial Dividend: 100100001
    [DIV] Divisor:          1101
    [DIV] 1001 XOR 1101 -> 1000 (pulled down 0)
    [DIV] 1000 XOR 1101 -> 1010 (pulled down 0)
    [DIV] 1010 XOR 1101 -> 1110 (pulled down 0)
    [DIV] 1110 XOR 1101 -> 0110 (pulled down 0)
    [DIV] 0110 XOR 00000000 -> 1101 (pulled down 1)
    [DIV] Final Step: 1101 XOR 1101 -> 000

Final Remainder: 000
[RESULT] SUCCESS: The remainder is zero. No errors detected.

>>> SCENARIO 2: Transmission WITH errors (Fault Simulation)
(Simulating noise... flipping a bit: 100100001 -> 101100001)
--- RECEIVER SIDE ---
Received Codeword: 101100001

Performing Modulo-2 Division on Received Data:
    [DIV] Initial Dividend: 101100001
    [DIV] Divisor:          1101
    [DIV] 1011 XOR 1101 -> 1100 (pulled down 0)
    [DIV] 1100 XOR 1101 -> 0010 (pulled down 0)
    [DIV] 0010 XOR 000000 -> 0100 (pulled down 0)
    [DIV] 0100 XOR 0000000 -> 1000 (pulled down 0)
    [DIV] 1000 XOR 1101 -> 1011 (pulled down 1)
    [DIV] Final Step: 1011 XOR 1101 -> 110

Final Remainder: 110
[RESULT] ERROR DETECTED: The remainder is non-zero. The data was corrupted during transmission.
```

3.2 Hamming Code Simulation Output

```
--- SENDER SIDE ---
Original Data: 1011001
Data Bits (m): 7
Redundant Parity Bits required (r): 4
Data with empty parity bits: 10101000100
Calculating Parity Bits (Even Parity):
  P1 checks: pos 1 (val 0), pos 3 (val 1), pos 5 (val 0), pos 7 (val 1), pos 9 (val 1), pos 11 (val
  ↪  1)
  => P1 is set to 0
  P2 checks: pos 2 (val 0), pos 3 (val 1), pos 6 (val 0), pos 7 (val 1), pos 10 (val 0), pos 11 (val
  ↪  1)
  => P2 is set to 1
  P4 checks: pos 4 (val 0), pos 5 (val 0), pos 6 (val 0), pos 7 (val 1)
  => P4 is set to 1
  P8 checks: pos 8 (val 0), pos 9 (val 1), pos 10 (val 0), pos 11 (val 1)
  => P8 is set to 0

Transmitted Codeword: 10101001110

```

```
>>> Simulating transmission noise...
Original sent: 10101001110
Received     : 10100001110

--- RECEIVER SIDE: Error Checking ---
Recalculating parity bits over received data:
  P1 parity check evaluates to: 1
  P2 parity check evaluates to: 1
  P4 parity check evaluates to: 1
  P8 parity check evaluates to: 0

[RESULT] Syndrome is 0111. Error detected at position 7 (from the right).
-> Fixing error... Flipping bit at index 4 (which is currently '0')
-> Corrected Data Codeword: 10101001110
```

4 Discussion

The simulations provided a detailed operational view of both error-handling mechanisms. During the CRC analysis, the modulo-2 division was observed step-by-step. It was verified that appending the calculated remainder (`001`) to the original data resulted in a codeword (`100100001`) that became perfectly divisible by the polynomial (`1101`), yielding a final remainder of `000` at the receiver. When an intentional fault was introduced (flipping the 3rd bit), the division logic was disrupted, producing a non-zero remainder (`110`), thereby successfully alerting the system to the corruption.

In the Hamming Code analysis, the capability to not only detect but correct a fault was demonstrated. Four parity bits ($P1, P2, P4, P8$) were utilized to secure the 7-bit data block. Upon intentionally corrupting the 7th bit from the right, the receiver's recalculation of the parities yielded a syndrome of `0111` (decimal 7). This syndrome accurately pinpointed the precise location of the error, allowing the algorithm to automatically flip the bit and reconstruct the original transmitted codeword (`10101001110`) without requiring data retransmission.

5 Conclusion

The design and implementation of error detection and correction mechanisms were successfully achieved using Python simulations. The CRC method proved highly efficient for detecting errors utilizing modulo-2 arithmetic, making it ideal for network protocols where retransmission is acceptable. Conversely, the Hamming Code implementation successfully localized and rectified single-bit errors through parity synchronization. This confirmed that while Hamming Code introduces a larger logarithmic overhead, its forward error correction (FEC) capabilities are invaluable in scenarios where retransmission is costly or impossible.

References

[1] GeeksforGeeks, "Cyclic redundancy check and modulo-2 division," 2025, accessed: 2026-04-18. [Online]. Available: https://www.geeksforgeeks.org/dsa/modulo-2-binary-division/

[2] R. W. Hamming, "Error detecting and error correcting codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950.

[3] GeeksforGeeks, "Hamming code implementation in python," 2025, accessed: 2026-04-18. [Online]. Available: https://www.geeksforgeeks.org/python/hamming-code-implementation-in-python/